UNIVERSIDAD
DE BURGOS

Automatización con Bash y Python

Practica_1

Alumno: Fernando Morán Fernández, con DNI 71378005N

Grado en Tecnologías Digitales para la Empresa

Profesor: Jaime Andres Ricon Aragon

En Miranda de Ebro, a 06 de marzo de 2025

ÍNDICE

Introducción 3
Objetivos 3
Marco teórico 3
Diseño y Metodología 4
Implementación 5
Resultados y pruebas 6
Discusión 8
Conclusiones 8
Manual de Ejecución 8
Repositorio GIT 9
Bibliografía 9

Automatización con Bash y Python

Introducción

El objetivo de esta práctica es aplicar los conceptos de automatización de tareas y gestión de procesos en el entorno de Linux. Para ello, se ha desarrollado un sistema de capturas de datos de sensores ambientales (temperatura y gases) mediante el protocolo MQTT, procesar dichos datos en formato JSON y generar visualizaciones gráficas tanto en la terminal como en archivos de imagen. Este proyecto combina el uso de scripts en Bash para el control del flujo de ejecución y scripts en Python para el procesamiento y análisis de datos.

Objetivos

- Comprender y aplicar la ejecución de procesos en segundo plano en Linux.
- Automatizar el control de procesos (inicio y terminación mediante señales/PID usando scripts de Bash).
- Procesar y extraer información estructurada a partir de archivos de registro (logs) que contienen datos en formato JSON.
- Generar representaciones visuales de los datos obtenidos (gráficas ASCII y exportación a PNG).
- Familiarizarse con el control de versiones usando Git y GitHub.

Marco teórico

Para el desarrollo de la práctica se han utilizado los siguientes conceptos fundamentales:

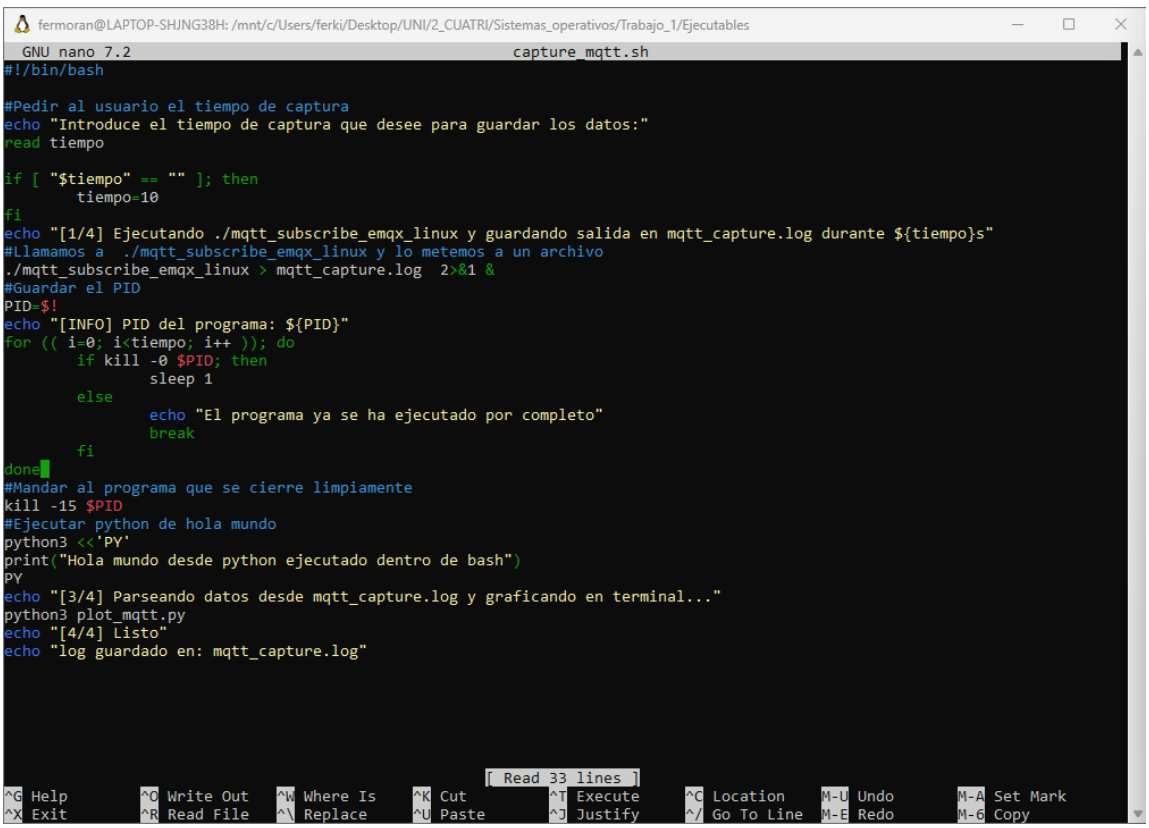
- **Gestión de procesos (PID):** Conocer este PID permite enviar señales al proceso (por ejemplo, Kill) para detenerlo de forma controlada.

- **Protocolo MQTT:** Es un protocolo de mensajería ligero, utilizado en el ámbito del internet de las Cosas (IoT) para transmitir datos de sensores.
- **JSON:** (JavaScript Object Notation) Es un formato de texto estándar y ligero utilizado para el intercambio de datos.

Diseño y Metodología

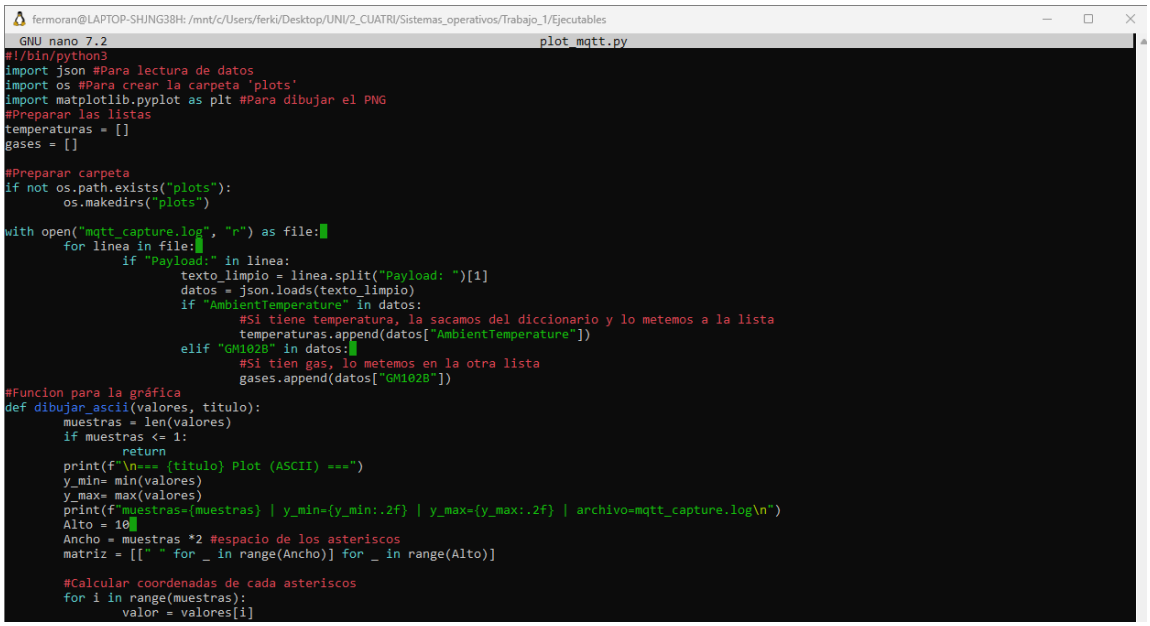
El enfoque seguido divide la práctica en dos bloques lógicos, orquestados por script principal:

- **Módulo de Captura (Bash):** El script capture_mqtt.sh actúa como coordinador. Pide al usuario el tiempo de ejecución, ejecuta el MQTT en segundo plano, guarda su PID, espera el tiempo indicado y, finalmente, detiene el proceso de forma limpia. Los datos recibidos se vuelcan a un archivo de registro llamado mqtt_capture.log.



- **Módulo de Análisis (Python):** Una vez terminada la captura, el script de Bash llama a plot_mqtt.py. Este script lee el archivo de

registro línea por línea, busca la etiqueta “Payload” para extraer las cadenas JSON, procesa los valores de temperatura y gas, y genera las gráficas de salida.




```
#!/bin/python3
import json #Para lectura de datos
import os #Para crear la carpeta 'plots'
import matplotlib.pyplot as plt #Para dibujar el PNG
#Preparar las listas
temperaturas = []
gases = []

#Preparar carpeta
if not os.path.exists("plots"):
        os.makedirs("plots")

with open("mqtt_capture.log", "r") as file:
        for linea in file:
                if "Payload:" in linea:
                        texto_limpio = linea.split("Payload: ")[1]
                        datos = json.loads(texto_limpio)
                        if "AmbientTemperature" in datos:
                                #Si tiene temperatura, la sacamos del diccionario y lo metemos a la lista
                                temperaturas.append(datos["AmbientTemperature"])
                        elif "GM102B" in datos:
                                #Si tien gas, lo metemos en la otra lista
                                gases.append(datos["GM102B"])
#Funcion para la gráfica
def dibujar_ascii(valores, titulo):
        muestras = len(valores)
        if muestras <= 1:
                return
        print(f"\n=== {titulo} Plot (ASCII) ===")
        y_min= min(valores)
        y_max= max(valores)
        print(f"muestras={muestras} | y_min={y_min:.2f} | y_max={y_max:.2f} | archivo=mqtt_capture.log\n")
        Alto = 10
        Ancho = muestras *2 #espacio de los asteriscos
        matriz = [[" " for _ in range(Ancho)] for _ in range(Alto)]

        #Calcular coordenadas de cada asteriscos
        for i in range(muestras):
                valor = valores[i]
```

Implementación

La solución se ha implementado mediante dos archivos clave:

- **Bash (capture_mqtt.sh):** Se utiliza la variable especial $! Para captura el PID del último proceso ejecutado en segundo plano y el comando sleep para la temporización.

```
fermoran@LAPTOP-SHJNG38H: /mnt/c/Users/ferki/Desktop/UNI/2_CUATRI/Sistemas_operativos/Trabajo_1/Ejecutables
  GNU nano 7.2                                   capture_mqtt.sh
#!/bin/bash

#Pedir al usuario el tiempo de captura
echo "Introduce el tiempo de captura que desee para guardar los datos:"
read tiempo

if [ "$tiempo" == "" ]; then
        tiempo=10
fi
echo "[1/4] Ejecutando ./mqtt_subscribe_emqx_linux y guardando salida en mqtt_capture.log durante ${tiempo}s"
#Llamamos a  ./mqtt_subscribe_emqx_linux y lo metemos a un archivo
./mqtt_subscribe_emqx_linux > mqtt_capture.log  2>&1 &
#Guardar el PID
PID=$!
echo "[INFO] PID del programa: ${PID}"
for (( i=0; i<tiempo; i++ )); do
        if kill -0 $PID; then
                sleep 1
        else
                echo "El programa ya se ha ejecutado por completo"
                break
        fi
done
#Mandar al programa que se cierre limpiamente
kill -15 $PID
#Ejecutar python de hola mundo
python3 <<'PY'
print("Hola mundo desde python ejecutado dentro de bash")
PY
echo "[3/4] Parseando datos desde mqtt_capture.log y graficando en terminal..."
python3 plot_mqtt.py
echo "[4/4] Listo"
echo "log guardado en: mqtt_capture.log"

                                        [ Read 33 lines ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut       ^T Execute   ^C Location  M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify   ^/ Go To Line M-E Redo      M-6 Copy
```

- **Python (plot_mqtt.py):** Se utiliza la librería estándar json para la lectura de datos y la librería externa matplotlib.pyplot para la generación de las imágenes PNG.

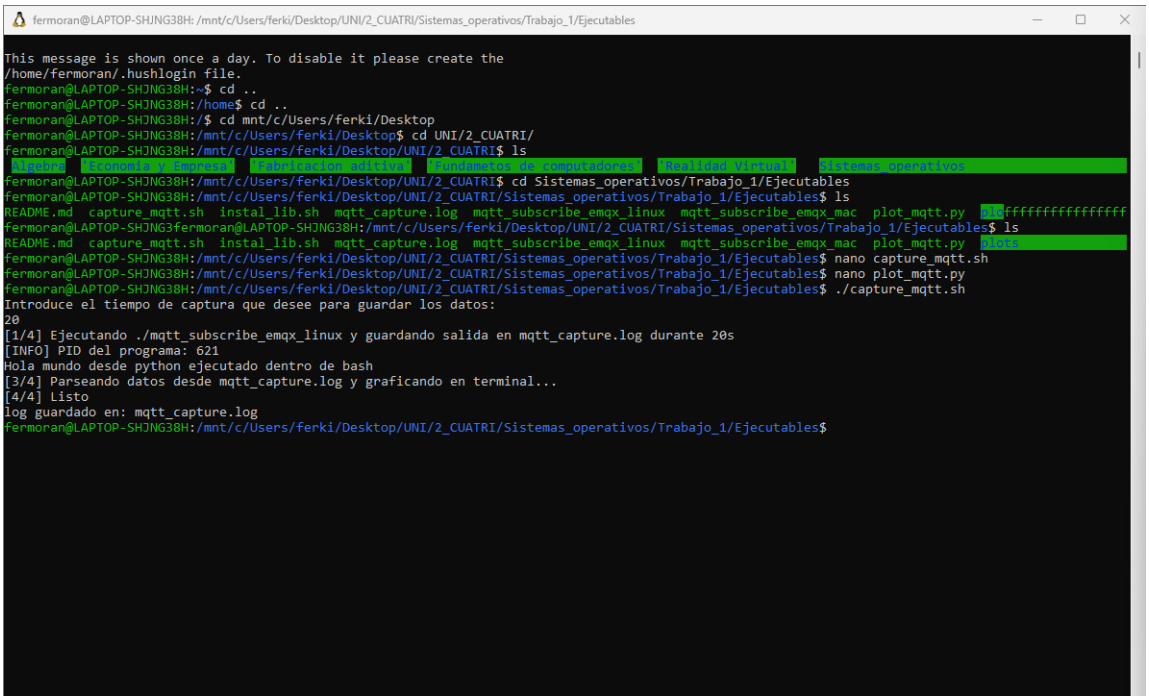
```
fermoran@LAPTOP-SHJNG38H: /mnt/c/Users/ferki/Desktop/UNI/2_CUATRI/Sistemas_operativos/Trabajo_1/Ejecutables
  GNU nano 7.2                                   plot_mqtt.py
#!/bin/python3
import json #Para lectura de datos
import os #Para crear la carpeta 'plots'
import matplotlib.pyplot as plt #Para dibujar el PNG
#Preparar las listas
temperaturas = []
gases = []

#Preparar carpeta
if not os.path.exists("plots"):
        os.makedirs("plots")

with open("mqtt_capture.log", "r") as file:
        for linea in file:
                if "Payload:" in linea:
                        texto_limpio = linea.split("Payload: ")[1]
                        datos = json.loads(texto_limpio)
                        if "AmbientTemperature" in datos:
                                #Si tiene temperatura, la sacamos del diccionario y lo metemos a la lista
                                temperaturas.append(datos["AmbientTemperature"])
                        elif "GM102B" in datos:
                                #Si tien gas, lo metemos en la otra lista
                                gases.append(datos["GM102B"])
```

Resultados y pruebas

El programa funciona según lo esperado, marcando alguna falta de datos cuando no están los sensores captando datos.

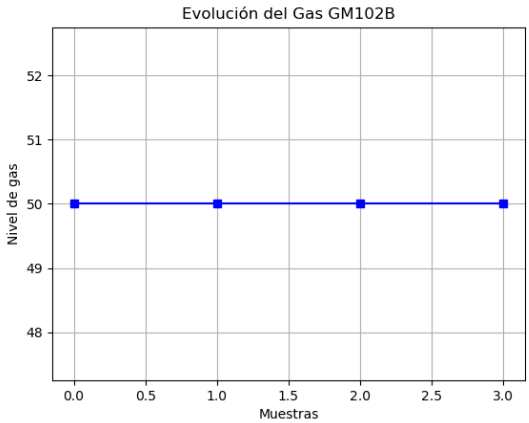
- ## Prueba de ejecución en terminal:

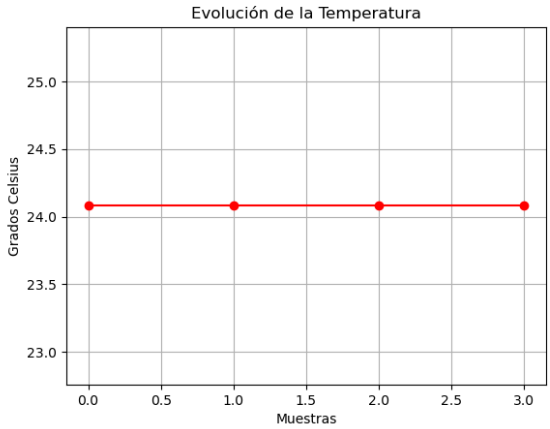
  Como se puede observar en la siguiente imagen, el programa no muestra la gráfica ASCII. Esto se debe a que, en el momento de la captura, los sensores no se encontraban emitiendo datos.



- ## Gráficas generadas:

  A continuación, se muestran las gráficas de gas y temperatura generadas correctamente y exportadas a PNG durante las pruebas de funcionamiento. (son cuando hice las pruebas y funcionaba los sensores)





Discusión

Durante la fase de pruebas, se detecta que, dependiendo del tiempo de ejecución, el script finalizaba antes de que los sensores remotos tuvieran tiempo de enviar el primer paquete de datos completo. Además de desconexiones de los sensores, con lo cual, impedían los registros de los datos.

Conclusiones

En la práctica se intenta demostrar la eficiencia de combinar Bash y Python en entornos Linux. Bash interactúa muy bien con el sistema operativo y la gestión del ciclo en la vida de los procesos. Mientras que Python ofrece potencia y simplicidad para el análisis de cadenas de texto (JSON) y la representación gráfica.

Manual de Ejecución

Para compilar y ejecutar la práctica:

- **Requisitos:** Sistema operativo Linux, bash, python3.
- **Dependencias:** Librería matplotlib (sudo apt install python3-matplotlib).
- **Ejecución:** Otorgar permisos (chmod +x capture_mqtt.sh) y ejecutar mediante ./capture_mqtt.sh

Repositorio GIT

Todo el código, historiales de cambios y documentación están alojados en el siguiente repositorio de GitHub:
Enlace: https://github.com/Fer851/Practicas_SO_2025_2026

Bibliografía

- Documentación oficial de Python (Módulo JSON):
  7. Entrada y salida — documentación de Python - 3.14.3
- Documentación oficial de Matplotlib:
  Pyplot tutorial — Matplotlib 3.10.8 documentation
- Material de apoyo de la asignatura